CONTENT

1 PURPOSE 2
2 SCOPE 2
3 RESPONSIBILITIES 2
4 TOOL IDENTIFICATION 2
5 INTENDED USE AND LIMITS OF USE 3
6 DETECTION METHODOLOGY 3
6.1 Events and import 4
6.2 Event catalog and severity 5
6.3 Detection rules 5
6.4 Alert life cycle 5
6.5 Worked example 6
7 SECURITY OF THE TOOL 6
8 TOOL CLASSIFICATION AND ERROR DETECTION 7
9 TOOL VALIDATION 8
10 TEST PLAN 9
11 TEST REPORT 11
12 CONFIGURATION AND CHANGE MANAGEMENT 12
13 SUPPLY CHAIN AND INTEGRITY 12
14 KNOWN LIMITATIONS 12
15 USAGE INSTRUCTIONS AND TRAINING 13
16 PERIODIC REVIEW AND RE-EVALUATION 13
17 EVIDENCE RECORD 13
17.1 Approval 14
18 DOCUMENTS & RECORDS 14
19 REVISION HISTORY 14

1 PURPOSE

This record documents the management of the tool **Fleet IDS SIEM** that supports the evaluation of intrusion detection system (IDS) events reported by vehicles. It shows how the tool is identified, which use it is intended for, how its influence on cybersecurity was assessed, how it is validated and kept under configuration control, and which limitations users have to know. It serves as evidence for the tool management requirements of ISO/SAE 21434 clause 5 (organizational cybersecurity management) for a tool used in the post-development activities (continual cybersecurity activities, clause 8, and operations and maintenance, clause 13).

> **Note:** The classification in chapter 8 and the validation criteria in chapters 9 and 10 are a **proposal of the tool owner**. They become effective when the cybersecurity process owner has reviewed and approved this record (chapter 17). Clause references are to be confirmed by the cybersecurity process owner.

2 SCOPE

The record applies to the in-house developed tool listed in chapter 4 and to the Python runtime and libraries it needs. It covers its use for the analysis of IDS events of INEOS vehicles.

- **In scope:** Fleet IDS SIEM (web application on the analyst PC, database, import, detection rules, case handling, audit log), the MCP interface for AI agents (`agent/mcp_server.py`) and the test scripts.
- **Not in scope:** the vehicle IDS and the CycurGUARD backend (data sources), the operating system, the browser, the authenticator apps and the Python runtime itself. They are managed by their owners and by IT and are only listed where the tool depends on them. The incident-response and vulnerability-management process that uses the alerts.

3 RESPONSIBILITIES

Role	Responsibility
Tool owner (Matthias Wachs)	Develops and maintains the tool, keeps this record current, performs the validation before each release, decides on tool changes.
Administrator of the installation	Creates accounts and assigns roles, sets the sign-in policy and detection parameters, protects the data folder, runs the verification of the audit log.
Analyst	Triages alerts, works cases, classifies event types, and verifies the findings with the system or component owners. Reports tool errors to the tool owner.
Cybersecurity process owner	Approves this record and the tool classification, decides whether a re-evaluation is needed.

4 TOOL IDENTIFICATION

Item	Fleet IDS SIEM
Programme	`idssiem.py` with the package `ids_siem/`, the web interface `static/` and the AI agent interface `agent/mcp_server.py`

Item	Fleet IDS SIEM
Purpose	Import and store IDS events, classify them by severity, detect patterns across the fleet (six rules), manage alerts and cases with checklist, experts and report, keep an audit log.
Version	build 720fa6a (the application has no version number; identified by the last commit that changed the tool: `720fa6a`)
Origin	In-house development, repository https://github.com/rng0xff/idssiem
Delivery form	Source code; Python 3.9 or later and the standard library. `openpyxl` only for Excel import. No installation, no network access needed; binds to the loopback address.
Inputs	CSV, XLSX, JSON exports of vehicle IDS reports; CycurGUARD backend (REST, optional); JSON via `POST /api/ingest`
Outputs	Alerts, cases, case report, CSV export of events and of the audit log, MCP answers for an AI agent
Data storage	SQLite file (`data/siem.db`, `IDS_SIEM_DB`), readable by the owner only

5 INTENDED USE AND LIMITS OF USE

The tool supports the analyst in finding and documenting conspicuous IDS events of the vehicle fleet. It **proposes** alerts; the analyst decides on status, suppression, case and response.

- The tool is used **only** to evaluate and document IDS events. It does not change anything in the vehicle, the IDS configuration or the backend; the backend is only read.
- Severity and category per event type are **defaults** of the event catalog (a starting point). They are adjusted to the TARA by the analyst; the tool does not know the real impact on the vehicle.
- A **missing alert is not proof of absence of an attack**: the rules find patterns in the data that was imported (chapter 14).
- Alerts, cases and exports are valid only together with the data that was imported and the tool version recorded in this record.
- The AI agent interface can read and add notes marked `[AI agent]`. It cannot change a status, link, close or delete anything.

The detection rules are described in chapter 6; the security properties of the tool in chapter 7.

6 DETECTION METHODOLOGY

This chapter describes how an event reported by a vehicle becomes an alert and how an alert is handled. The description follows the code (`ids_siem/ingest.py`, `catalog.py`, `rules.py`) and the README of the tool.

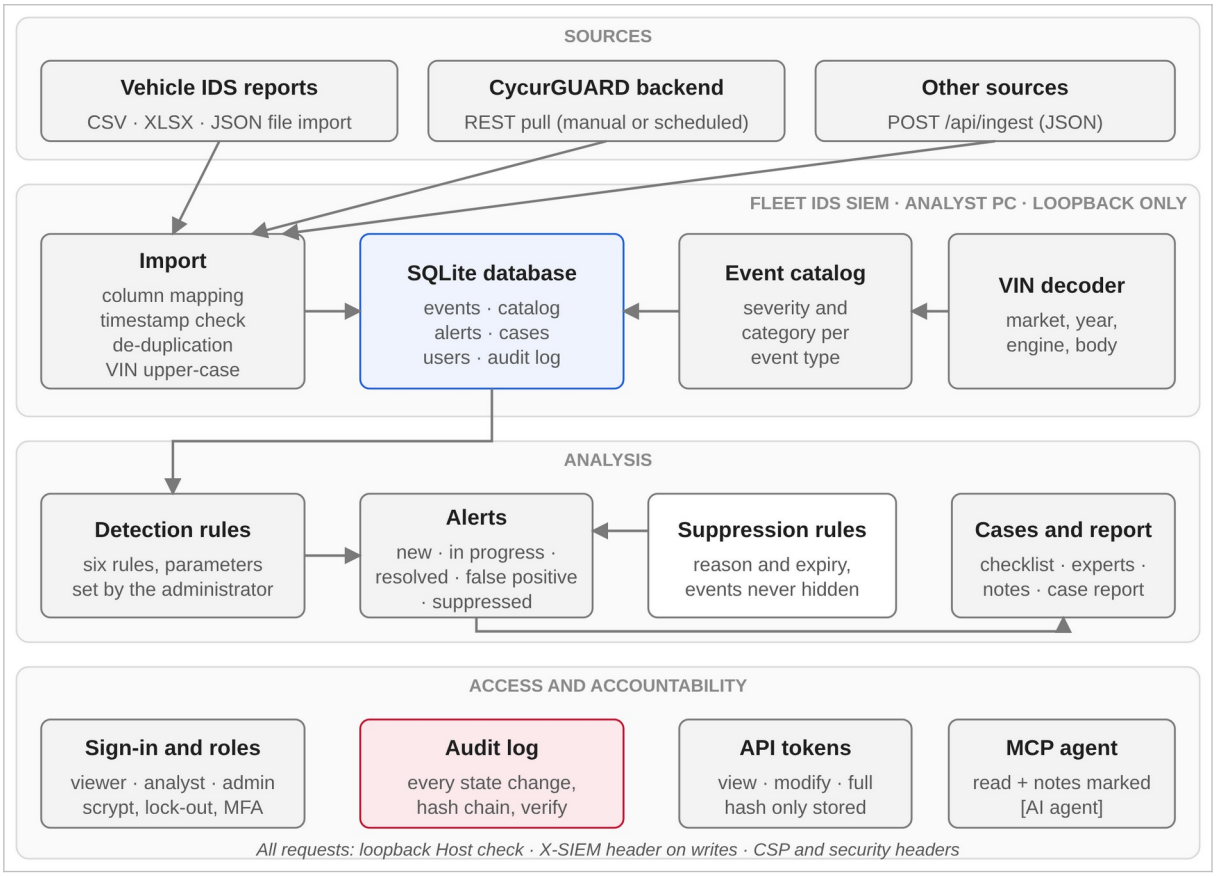


Figure 1: Overview: from the vehicle report to the case, with access control and audit log

6.1 Events and import

- **Event:** one IDS report with event type, event id, timestamp, VIN, ECU name and id, name and description. Column names of the INEOS / CycurGUARD exports are mapped automatically (aliases such as `report.event_type`, `sev_metadata.timestamp`).
- **Normalisation:** VIN in upper case, event type in lower case; a row without VIN or event type is invalid and only counted.
- **De-duplication:** a file row is identified by a hash of its content plus its n-th occurrence in the file. Importing the same data again adds nothing; genuinely repeated identical rows are kept. Backend events are identified by the backend event id.
- **Timestamp plausibility:** a timestamp before the cut-off date (default 2022-01-01) or more than one day in the future is treated as an unsynchronised ECU clock. Such events count everywhere but do not take part in the time-based rules (burst, multiple ECUs, fleet spike).
- **VIN decoder:** market, model year, body, engine, hand of drive and plant are derived from the VIN positions; the ISO 3779 check digit is verified. Unknown codes are listed, not guessed.

6.2 Event catalog and severity

Every event type (type and event id) has a catalog entry with short name, category and severity (info, low, medium, high, critical). The tool ships defaults for the known types of the IDS (for example `UDS_CRIT_NR_NRC` = high, `INVALID_SIG` = medium, `BURST_START` = info). An unknown type is registered as **unclassified** with severity low until an analyst classifies it. The severity of a type is the basis for all rules.

6.3 Detection rules

The rules run after every import or sync and on demand. Parameters can be changed by the administrator; the defaults are shown.

Rule	Condition	Severity of the alert
High-severity event	Event type with catalog severity at or above the chosen minimum (default high); grouped per vehicle, ECU and event type.	Catalog severity
Event burst	At least N events (default 3) of the same type on one vehicle within a sliding window (default 60 min). Overlapping windows form one cluster. Events of severity info and events without a valid timestamp are ignored.	Catalog severity raised by one level, at most high
Multiple ECUs	At least N different ECUs (default 3) report non-info events on one vehicle on the same UTC day.	High if one event is high or above, otherwise medium
Vehicle outlier	Number of non-info events of a vehicle is more than K standard deviations (default 3) above the fleet mean, at least 10 events, at least 5 vehicles in the data.	Medium
Fleet-wide spike	A UTC day with more non-info events than the daily mean plus K standard deviations (default 3), at least 30 events, at least 7 days of data. Spike days at most one quiet day apart form one alert.	Medium
Unclassified event type	An event type that is not in the built-in catalog and was not yet classified.	Low

6.4 Alert life cycle

- An alert has a stable key (rule and its scope). Running the rules again updates the alert (count, time range, severity) and **never creates it twice**; the triage state of an alert is kept.
- Status: new, in progress, resolved, false positive, suppressed. A resolved or false-positive alert is **re-opened** with a system note when new matching events arrive.
- **Suppression rules** (detection, ECU, event type, vehicle; at least one) with mandatory reason and expiry (default 90 days) mark matching new alerts as suppressed. Alerts already in work are not suppressed; events are never hidden. Creation and deletion are written to the audit log.
- An alert can be linked to a **case** with checklist, documented technical experts, notes and a case report. A case suggests related cases.

6.5 Worked example

One vehicle reports the event type `INVALID_SIG` (catalog severity medium) four times: at minute 0, 20, 45 and 120 (file `tests/data/worked_example.csv`). Burst rule with window 60 min and threshold 3:

Step	Rule applied	Result
Events 1, 2, 3	Event 3 is 45 min after event 1 (≤ 60 min): three events in the window = threshold	Cluster of **3**
Event 4	75 min after event 3, window of event 3 holds only itself	not part of the cluster
Severity	medium (catalog) raised by one level	**high**
Other rules	High-severity: type is medium, not high. Multiple ECUs: one ECU only.	no alert

Result: one alert "3× INVALID_SIG within 45 min", severity high, three events. This example is recomputed with the detection code at every release (last check: 2026-10-16) and is used as manual test case TC-M03 (chapter 10).

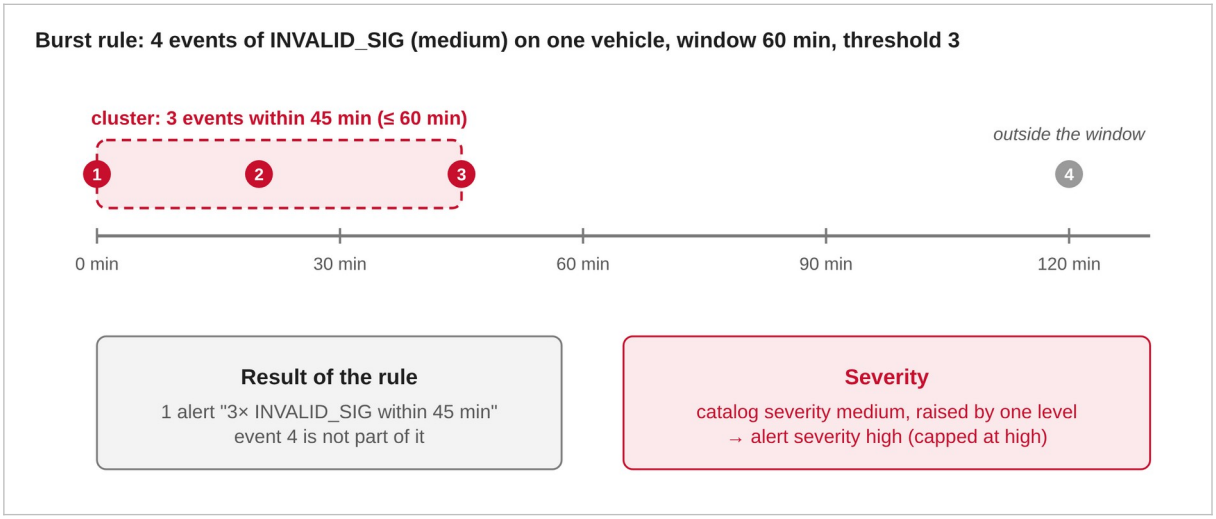


Figure 2: Worked example: events, burst cluster and resulting alert severity

7 SECURITY OF THE TOOL

The tool processes vehicle identification numbers and stores credentials of the analysts. The measures below are described in the README; the right-hand column names the automated test that checks them (chapter 11).

Area	Measure	Evidence
Network	Binds to 127.0.0.1 only; accepts only loopback `Host` headers; every write needs an `X-SIEM` header (cross-site requests cannot change data).	`security_test`
Browser	Strict content-security policy (no inline scripts), frame, referrer and sniffing protection; HttpOnly SameSite cookies; output escaped.	`security_test`

Area	Measure	Evidence
Authentication	scrypt password hashes; account lock for 5 minutes after 5 failures; global limit of failed sign-ins; same answer for unknown user and wrong password; sessions stored as hash.	`test_security`, `security_test`
Two-step sign-in	TOTP (RFC 6238), passkeys (WebAuthn), recovery codes stored as hash; can be required by the administrator.	`test_security` (RFC 4226 vectors); TC-M06
Roles and tokens	viewer / analyst / admin; API tokens with own permission and expiry, only hash stored; access = lower of token and role.	`security_test`, `test_mcp`
Audit log	Every state-changing action with person, channel, action and target; hash chain over all entries; verification function. Edit and deletion in the middle of the log are detected; rebuilding by someone with write access to the database file is not.	`test_security` (tamper and delete)
Input	Parameterised SQL; size limits; exported CSV neutralises formula cells.	`security_test`
Data at rest	Data folder and database readable by the owner only. Backend token and authenticator secrets are stored in clear text because they must be used: protect the file.	`security_test` (permissions)

8 TOOL CLASSIFICATION AND ERROR DETECTION

Influence on cybersecurity. The tool produces alerts and case records that the analyst reviews before they lead to a decision (for example a vulnerability or incident entry, a change of the TARA or a field action). It has no influence on the vehicle. The influence is therefore **indirect**: a missed or wrongly rated alert could delay a response. The measures below reduce the probability that such an error stays undetected.

Potential tool failure	Effect	Measure to prevent or detect it
Missed detection (false negative)	A pattern in the data is not reported	Rules are documented with thresholds (chapter 6) and tested at the threshold (`test_rules`); every event stays searchable and countable independent of the rules; fleet figures on the overview page; the analyst can search events directly.
False alarm	Effort, alert fatigue	Suppression rules with reason, expiry and audit entry; parameters tunable; events are never hidden by a suppression.
Event lost or doubled on import	Counts and rules distorted	Import result shows total / new / duplicate / invalid; tests for re-import, repeated rows and backend ids (`test_ingest`); compare with the source (TC-M02).
Wrong timestamp handling	Event missing in a burst or time range	Implausible timestamps are flagged and counted separately (shown with `ts:invalid`); tested (`test_ingest`).
Wrong severity of an event type	Alerts too low or too high	Catalog defaults are visible and editable on the Catalog page; unknown types are flagged for classification; changes are in the audit log.

Potential tool failure	Effect	Measure to prevent or detect it
Wrong vehicle attributes	Wrong grouping by market or engine	Decoder lists unknown codes instead of guessing; check digit verified; tested with the reference vector (`test_vin`, TC-M09).
Unauthorised access or change	Disclosure or manipulation of cases	Chapter 7; audit log; roles.
Unnoticed change of the tool	Tool behaves differently than validated	Configuration management and source-tree checksum (chapters 12 and 13).
Misuse of the AI agent interface	Wrong statements in cases	Agent can only add notes marked `[AI agent]`, cannot change status; every action is in the audit log; `test_mcp`.

Classification (proposal). Based on the indirect influence and the possibility to detect errors through the independent data (events stay available), the review by the analyst and the audit log, the tool is classified as a **tool with low impact, high error detection**. For this class the tool management consists of identification, controlled use, regression testing and configuration control as described in this record. A qualification beyond that is not planned. This is a proposal of the tool owner and is to be confirmed by the cybersecurity process owner.

9 TOOL VALIDATION

The tool is validated by an automated regression test that is run before every release and after every change of the source: `python tests/run_tests.py` (unit tests, no server needed) and `python tests/run_integration.py security|mcp` (tests against a real server on a temporary database). The release script runs all of them.

Test	What it verifies
test_ingest	Import: column aliases, CSV and JSON, normalisation of VIN and event type, de-duplication on re-import, repeated identical rows, backend import by id, timestamp plausibility, registration of unknown event types in the catalog.
test_rules	The six detection rules with their thresholds (high-severity, burst, multi-ECU, vehicle outlier, fleet spike, unclassified event type), suppression rules, alert re-opening, idempotent re-runs, disabled rules.
test_security	scrypt password hashing, account set-up, sign-in, lock-out after five failures, sessions stored as hash, role and username validation, last administrator protected, audit hash chain (modification and deletion detected), TOTP (RFC 4226 vectors).
test_vin	VIN decoder: ISO 3779 check digit (reference vector), EU/ROW and US/Canada decoding, malformed VINs, unknown codes.
security_test	Running application (temporary database, loopback): security headers and CSP, Host-header check, malformed requests, role enforcement, token permissions, CSV formula neutralisation, file permissions, sign-in flood limit.
test_mcp	MCP server for AI agents against a seeded server: tool list, read-only tools, note tools, permission of view and modify tokens, error handling, attribution and audit entry.

Acceptance criterion: all six test scripts end without error (exit code 0, 124 test cases in total) and every informational line they print is assessed (chapter 11). A failing test blocks the release.

Limits of the validation. The automated tests check the code with synthetic data and one reference vector. They do not use real vehicle data, the CycurGUARD backend (needs a backend; manual case TC-M08) or a browser (user interface is checked manually). Statistical rules (outlier, spike) are tested with constructed data; their usefulness for real fleet data is judged by the analysts. The tests do not replace the review of an alert by an analyst. The unit tests were added together with this record; before that, only the two API tests existed.

10 TEST PLAN

Objective. Show with repeatable tests that the tool imports, detects, stores and protects data as described in chapters 6 and 7 and that the failure modes of chapter 8 are covered. The plan applies to every release and to every re-evaluation (chapter 16).

Test items. The tool of the release under test (chapter 4).

Test environment. Automated tests: Python 3.13.16 (any Python 3.9 or later), standard library only, a temporary database per test; integration tests start the application on a free loopback port. Manual tests: a PC with the delivered source, a current browser and real IDS exports; an authenticator app; a CycurGUARD backend for TC-M08.

Test levels. (1) Unit tests of detection, import, VIN decoding, authentication, audit chain and TOTP. (2) Integration tests of the web interface and the MCP server. (3) Manual acceptance tests with real data.

Entry criteria. Source committed; working tree clean; no open defect that affects a detection, an import or the access control.

Pass / fail criteria. Automated: every script ends with exit code 0. Manual: actual result equals expected result. A release needs all cases passed; a failed case blocks the release until the defect is fixed and the whole plan is run again.

Automated test cases (run by the release script; results in chapter 11):

ID	Script	Objective and expected result	Failure modes (chapter 8)
TC-A01	test_ingest	Re-importing the same file adds nothing; identical rows inside one file are kept; rows without VIN or type are counted invalid; implausible timestamps are flagged; unknown event types become "unclassified"; a file without known headers is rejected.	Event lost, doubled or mis-timed on import
TC-A02	test_rules	Each rule raises an alert exactly when its documented condition holds (and not just below the threshold); severity bump and cap; suppression needs scope and reason; a resolved alert re-opens on new events; running the detections twice creates nothing twice.	Missed or false detection
TC-A03	test_security	Passwords verify only with the right value; unknown user and wrong password give the same answer; five failures lock the account; the audit chain reports an edited or deleted entry; the TOTP implementation reproduces the RFC 4226 test values.	Unauthorised access; undetected change of the audit log

ID	Script	Objective and expected result	Failure modes (chapter 8)
TC-A04	test_vin	Check digit equals the reference value; malformed VINs are reported, not decoded; an unknown code is listed and does not raise.	Wrong vehicle attributes
TC-A05	security_test	All checks of tools/security_test.py pass against a freshly started server: headers present, foreign Host headers refused (403), writes without X-SIEM header refused, roles and tokens enforced, exports neutralised, database file 0600, flood throttled.	Unauthorised access; weak web interface
TC-A06	test_mcp	A view token cannot write; a modify token can only add notes marked "[AI agent]" and cannot change status; invalid tokens and arguments give clear errors; the action is in the audit log.	Misuse of the AI agent interface

Manual test cases (executed by the tool owner before each release; the result is entered in `tools/release-report/manual-results.json`):

ID	Action	Expected result	Result (this version)
TC-M01	Start `python idssiem.py` on a PC without network connection, open http://localhost:8765, create the first administrator.	Application starts, no error, no network access needed; sign-in page then dashboard.	open
TC-M02	Import an original IDS report export (CSV and XLSX; `pip install -r requirements.txt` for XLSX). Compare the row counts of the import result with the source file; import the same file again.	Counts equal the source; second import adds no event (all duplicates).	open
TC-M03	Import `tests/data/worked_example.csv` (chapter 6) and run the detections (Data → Run detections).	Exactly one burst alert "3× INVALID_SIG within 45 min", severity high, 3 events; the fourth event is not part of it.	open
TC-M04	Detections page: raise the burst threshold to 4, run detections; then create a suppression rule for the alert with a reason.	Alert no longer reproduced at threshold 4; with the rule the alert shows status Suppressed with the reason in its history; events stay visible.	open
TC-M05	Create a case from an alert, work through a checklist step, document an expert, generate the case report.	Case, step status, expert and report are complete and attributed to the signed-in user.	open
TC-M06	Account: add an authenticator app (TOTP), sign out, sign in with password and code; then sign in with a recovery code.	Both ways work; each recovery code works once; the entries are in the audit log.	open
TC-M07	Audit log (administrator): verify integrity.	Chain reported as intact; the sign-ins and changes of the test are listed.	open

ID	Action	Expected result	Result (this version)
TC-M08	If a CycurGUARD backend is available: Users → Test connection, Data → Check connection, then a small sync (last days).	Connection and event count plausible; imported events carry backend details; a second sync adds nothing.	open
TC-M09	Open the Vehicles page for a VIN of a real vehicle.	Market, year, body, engine and hand of drive agree with the vehicle documentation.	open
TC-M10	On the installation used for analysis run `python tools/release_report.py --hash` and compare with the source-tree checksum of the evidence record.	Checksums equal.	open

Coverage of the failure modes. Missed or false detection: TC-A02, TC-M03, TC-M04. Data loss / duplication: TC-A01, TC-M02, TC-M08. Wrong vehicle attributes: TC-A04, TC-M09. Unauthorised access: TC-A03, TC-A05, TC-M06. Audit log integrity: TC-A03, TC-M07. Web interface: TC-A05, TC-M01. AI agent interface: TC-A06. Changed tool files: TC-M10.

Defects. A failed case is recorded with the commit, the data used and the actual result. After the fix the failed case and the complete automated test are repeated. If the defect could have hidden an alert, the tool owner informs the analysts and the affected period is re-evaluated.

Responsibilities and schedule. The tool owner prepares and executes the tests and records the results. The plan is executed before every release and on every trigger of chapter 16; the cybersecurity process owner approves changes of the plan.

11 TEST REPORT

Execution of the automated tests of the test plan (chapter 10) on the release baseline.

Item	Value
Executed	2026-10-16, 08:58 UTC
Source	Version build 720fa6a, HEAD 720fa6a16796dbef055ed7992a0221eccd2ff34f
Environment	Python 3.13.16, Linux, SQLite 3.45.1, openpyxl 3.1.5
Command	`python tools/release_report.py` (unit tests per module, `tests/run_integration.py security`, `tests/run_integration.py mcp`)
Overall result	**PASSED** – all six test scripts ended with exit code 0 (124 test cases)

Test	Result	Output (key lines)
test_ingest	Passed	14 test cases.
test_rules	Passed	23 test cases.
test_security	Passed	20 test cases.
test_vin	Passed	7 test cases.
security_test	Passed	34/34 passed.
test_mcp	Passed	26/26 passed.

The tests print no informational lines: every check either passes or fails the script.

The full console output is reproduced by running the commands above on the commit named in the evidence record.

12 CONFIGURATION AND CHANGE MANAGEMENT

- **Version control:** all source, tests and documents are in one Git repository. Every change is a commit with author and date; the history starts on 2026-10-14.
- **Identification:** the application has no version number of its own. A release is identified by the last commit that changed the tool (or a git tag, if set) and by the SHA-256 checksum over all tool files (`python tools/release_report.py --hash`, chapter 13).
- **Change procedure:** change → automated tests → commit → release record. Event catalog defaults and detection parameters are changed on request of the cybersecurity process owner or the analysts and are visible in the audit log of the installation.
- **Release checklist:** (1) `python tests/run_tests.py` passes, (2) manual test cases executed and entered, (3) the record is regenerated with `python tools/release_report.py --release`, which runs all tests and writes version, commit, checksum and test report into chapters 11 and 17, (4) the record is reviewed and signed (chapter 17).
- **Issue handling:** errors found by users are reported to the tool owner, assessed for their effect on alerts and cases already created, and corrected in a new version.

13 SUPPLY CHAIN AND INTEGRITY

The tool uses the Python standard library only, with one optional third-party package. The web interface (`static/`) loads no script, font or stylesheet from the network.

Component	Version	Licence	Use	Remark
openpyxl	3.1.5 (requirement: `openpyxl>=3.1`)	MIT	Run time, optional	Reads Excel imports only. The requirement is **not pinned** to an exact version.
Python	3.13.16 (3.9 or later required)	PSF	Run time	Provided by the installation, not by the tool.
SQLite	3.45.1	Public domain	Run time	Part of Python.

Integrity. The checksum over the 36 tool files (`ids_siem/`, `static/`, `agent/`, `idssiem.py`, `requirements.txt`) is recorded for each release (chapter 17). It can be recomputed on any installation with `python tools/release_report.py --hash`. The tool runs locally; imported data is not transmitted anywhere except the optional read access to the CycurGUARD backend that the administrator configures.

14 KNOWN LIMITATIONS

- The rules only see imported data. Events with an unsynchronised clock do not take part in time-based rules.

- Event catalog defaults are a starting point and not an assessment of the impact on the vehicle.
- The statistical rules (vehicle outlier, fleet spike) need at least 5 vehicles resp. 7 days of data and are not suitable for small data sets.
- The audit log is hash-chained: it detects edits and deletions inside the log, but someone with write access to the database file could rebuild it, and cutting off the newest entries goes unnoticed unless the latest hash was noted.
- The backend token and the authenticator secrets are stored in the database in clear text. Protect the database file (disk encryption, restricted account). Traffic is plain HTTP on the loopback address; the port must not be exposed to a network.
- Passkeys work only at `http://localhost:<port>`.
- The application has no version number, and `openpyxl` is not pinned to an exact version.
- The integration with the CycurGUARD backend is checked manually (TC-M08); the automated tests use no backend.

15 USAGE INSTRUCTIONS AND TRAINING

- The README of the repository describes installation, pages, rules, roles, sign-in, audit log and security properties; `agent/README.md` the AI agent interface.
- New analysts are introduced by the tool owner on the analysis PC and work through a training case (import, alerts, suppression, case, report) before they handle real alerts.

16 PERIODIC REVIEW AND RE-EVALUATION

This record is reviewed at least once a year and re-evaluated, including a complete test run, when one of the following happens:

- the tool is changed after the last release,
- the format of the IDS report or the CycurGUARD interface changes,
- the Python version or openpyxl is changed on the analysis PC,
- an error in the tool is found that hid or distorted an alert,
- the incident-response process or the tool management requirements change.

17 EVIDENCE RECORD

Baseline for revision 001 of this record. For a new release, repeat the entries and add a row to the revision history.

Item	Value
Tool version	build 720fa6a

Item	Value
Source commit (last change of the tool)	720fa6a16796dbef055ed7992a0221eccd2ff34f
Commit date	2026-10-16T10:58:04+02:00
Date of validation	2026-10-16
Test environment	Python 3.13.16, Linux; 6 test scripts, 124 test cases
Test result	All 6 test scripts passed (test_ingest, test_rules, test_security, test_vin, security_test, test_mcp)
Worked example (chapter 6)	Verified with the detection code: burst alert "3× INVALID_SIG within 45 min", severity high, 3 events
SHA-256 of the tool files (36 files)	2547ad917e6fe903c88f497aee1fc55af6b8f89e8414e5c5fb7445f735ebdfe2
SHA-256 idssiem.py	760565f5e6858a9cef7f714206d364608f6d2fd7d524c758abb1b8cef4728fc9
SHA-256 static/index.html	5afee84fa8b4c854ac11004da8fd12573ee3f55333877e0c54f1d0dfc5a3d468
Manual test cases TC-M01 to TC-M10 (chapter 10)	0 of 10 passed for this version – the open cases are executed and entered by the tool owner at the release (tools/release-report/manual-results.json)

17.1 Approval

Role	Name	Signature	Date
Tool owner	Matthias Wachs		
Cybersecurity process owner			

18 DOCUMENTS & RECORDS

#	Description
1	ISO/SAE 21434:2021, clause 5 (organizational cybersecurity management, tool management), clause 8 (continual cybersecurity activities) and clause 13 (operations and maintenance)
2	Fleet IDS SIEM README and agent/README.md
3	Source repository https://github.com/rng0xff/idssiem (history, tests, release script)
4	Tool Management Record – TARA Tools (same format, for the TARA Workbench)

19 REVISION HISTORY

Revision	Description of Change	Editor	Date
001	Creation of document	Matthias Wachs	16.10.2026
002	Re-validation for build 720fa6a (commit 720fa6a); test report, checksums and evidence updated	Matthias Wachs	16.10.2026